Community Help Board & Emergency Network

DBMS Lab Project Report

A CRUD-based Web Application built with PHP & MySQL

Course	CSE 2208 — Database Management System Lab
Student Name	Rifat Al Araf
Student ID	242071066
Section	A
Submission Date	21 July 2026
Course Instructor	Tousif Hasan Lavlu

GitHub Repository: https://github.com/rifat07yoo-hash/Community-Help-Board

Tech Stack: PHP 8, MySQL, PDO, HTML5, TailwindCSS, Vanilla JavaScript

1. Introduction

The **Community Help Board & Emergency Network** is a full-stack web application that lets community members post and coordinate emergency help requests — food, blood, medical supplies, shelter, and general donations — in real time. Registered users can create requests, track collection progress toward a target quantity, comment publicly on a request, message the requester privately, rate and review helpers, and browse a directory of registered blood donors. A moderation layer lets administrators ban abusive users, delete posts, and clear report flags.

This report documents the system built to satisfy the CSE 2208 DBMS Lab requirement: a project that implements full **CRUD (Create, Read, Update, Delete)** operations against a **MySQL** relational database, with any web stack of the student's choice. The application is implemented with a **PHP backend** (PDO prepared statements throughout) and a **MySQL** database of seven normalized tables, with a Tailwind/vanilla-JavaScript frontend consuming a JSON REST API.

2. Objectives

- Design a normalized relational schema in MySQL that models users, help requests, comments, private messages, notifications, ratings, and reports.
- Implement complete CRUD functionality for the core entity (help requests) and supporting entities (comments, ratings, reports) using parameterized SQL queries.
- Build a secure authentication system with hashed passwords and server-side session management.
- Enforce authorization rules server-side (a user may only edit/delete their own content; only admins may moderate).
- Provide a usable, responsive interface for posting, searching, filtering, and coordinating help requests.
- Track project progress through weekly milestones and GitHub commit history, per the course rubric.

3. Requirement Analysis

Per the course announcement, the project had to satisfy the following requirements:

Requirement	How it is satisfied
CRUD operations using MySQL	**api/requests.php** implements full Create/Read/Update/Delete for help requests; comments, ratings, reports, and admin actions add further CRUD endpoints, all via PDO prepared statements against MySQL.
Any web development stack/framework	PHP 8 (no framework) for the backend API; HTML5 + TailwindCSS + vanilla JavaScript (fetch API) for the frontend.
Weekly progress review (10%)	Tracked via iterative development milestones (see Section 9 — Development Timeline).
GitHub commit history & contribution (5%)	Project maintained in a Git repository with incremental, descriptive commits per feature/module.
Project report (5%)	This document.

Requirement	How it is satisfied
Final presentation & demonstration (5%)	Live demo of registration, posting, editing, messaging, rating, and admin moderation.
Project features & functionality (15%)	See Section 5 — Features Implemented.

Note: this project was originally prototyped with Firebase/Firestore (a NoSQL, client-side database), which does not satisfy the MySQL requirement. It was subsequently rebuilt end-to-end on a PHP + MySQL stack, preserving the same feature set while moving all data access behind server-side, parameterized SQL queries.

4. System Architecture

The system follows a classic three-tier architecture: a browser-based presentation layer, a PHP application layer exposing a JSON REST API, and a MySQL data layer accessed exclusively through PDO prepared statements.

Layer	Technology	Responsibility
Presentation	HTML5, TailwindCSS, JavaScript (fetch)	Renders the UI; calls the API; never talks to the database directly.
Application / API	PHP 8 (PDO)	Validates input, enforces authentication/authorization, executes SQL, returns JSON.
Data	MySQL 8 / MariaDB	Stores all persistent data in 7 normalized, foreign-key-constrained tables.

Request Flow Example — Creating a Help Request

- User fills the "Post Emergency Request" form and submits (optionally attaching an image).
- JavaScript sends a `multipart/form-data` POST to `api/requests.php` with `action=create`.
- PHP validates the session, validates and sanitizes each field, stores the image via `includes/upload.php`, and runs a parameterized `INSERT` into `help_requests`.
- A row is also inserted into `notifications` to broadcast the new request.
- The frontend re-fetches the request list and re-renders the feed.

5. Features Implemented

Module	Features
Authentication	Register, login, logout; bcrypt password hashing; server-side sessions; banned-user lockout.
Help Requests	Create / edit / delete own requests; category & priority tagging; target vs. collected quantity progress bar; image upload; mark resolved/reopen; search and filter by category, priority, resolved status, and volunteer-only posts.
Comments	Public discussion thread per request; add, edit, and delete own comments.
Private Messaging	Per-request coordination chat between requester and helper; unread badges; inbox view of all active threads; delete own sent messages.
Notifications	Global activity feed of new/updated/resolved requests; unread counter; clear all.
Ratings & Reviews	5-star rating with optional written review, one review per rater–target pair (upsert); average rating shown on profile and donor cards.
Blood Donor Directory	Auto-generated list of users who registered a blood group, with one-tap call and review actions.
Reporting & Moderation	Users can flag a post once; posts with 3+ flags auto-hide from the public feed; admins can delete posts, clear flags, and ban/unban or delete users.
Public Profiles	View any user's name, location, blood group, rating, and post history.
Maps & Sharing	Embedded Google Map preview per request, one-tap directions, and clipboard-based share text.

6. Database Design

The schema (`schema.sql`) consists of seven tables in third normal form, linked by foreign keys with `ON DELETE CASCADE` so that deleting a user or a request automatically cleans up its dependent rows (comments, messages, reports, ratings).

6.1 Entity-Relationship Overview

- One **user** can create many **help_requests** (1:N).
- One **help_request** can have many **comments** and many **messages** (1:N each).
- Two **users** exchange **messages** that are always scoped to one **help_request** (M:N via messages, resolved per request).
- A **user** can **rate** another **user** at most once (unique constraint on rater–target pair).
- A **user** can **report** a given **help_request** at most once (unique constraint on reporter–request pair).

6.2 Table Definitions

users

Column	Type	Constraints / Notes
id	`INT`	PK, AUTO_INCREMENT
name	`VARCHAR(100)`	NOT NULL
email	`VARCHAR(150)`	NOT NULL, UNIQUE
password	`VARCHAR(255)`	bcrypt hash (password_hash)
phone	`VARCHAR(20)`	
location	`VARCHAR(255)`	
blood_group	`ENUM(...)`	A+, A-, B+, B-, O+, O-, AB+, AB-, Unknown
is_volunteer	`TINYINT(1)`	DEFAULT 0
is_admin	`TINYINT(1)`	DEFAULT 0
is_banned	`TINYINT(1)`	DEFAULT 0
profile_image	`VARCHAR(255)`	NULLABLE, path under /uploads
created_at	`TIMESTAMP`	DEFAULT CURRENT_TIMESTAMP

help_requests

Column	Type	Constraints / Notes
id	`INT`	PK, AUTO_INCREMENT
user_id	`INT`	FK → users(id), ON DELETE CASCADE
category	`ENUM`	food, blood, medical, shelter, other
priority	`ENUM`	urgent, high, medium
location	`VARCHAR(255)`	NOT NULL
contact	`VARCHAR(50)`	NOT NULL
description	`TEXT`	NOT NULL
target_qty / collected_qty	`INT`	progress-bar quantities
image	`VARCHAR(255)`	NULLABLE
resolved	`TINYINT(1)`	DEFAULT 0
created_at / updated_at	`TIMESTAMP`	auto-managed

comments

Column	Type	Constraints / Notes
id	`INT`	PK, AUTO_INCREMENT
request_id	`INT`	FK → help_requests(id), CASCADE

Column	Type	Constraints / Notes
user_id	`INT`	FK → users(id), CASCADE
text	`TEXT`	NOT NULL
created_at	`TIMESTAMP`	DEFAULT CURRENT_TIMESTAMP

messages

Column	Type	Constraints / Notes
id	`INT`	PK, AUTO_INCREMENT
request_id	`INT`	FK → help_requests(id), CASCADE
sender_id / recipient_id	`INT`	FK → users(id), CASCADE
text	`TEXT`	NOT NULL
is_read	`TINYINT(1)`	DEFAULT 0
created_at	`TIMESTAMP`	DEFAULT CURRENT_TIMESTAMP

notifications

Column	Type	Constraints / Notes
id	`INT`	PK, AUTO_INCREMENT
text	`VARCHAR(500)`	NOT NULL
is_read	`TINYINT(1)`	DEFAULT 0
created_at	`TIMESTAMP`	DEFAULT CURRENT_TIMESTAMP

ratings

Column	Type	Constraints / Notes
id	`INT`	PK, AUTO_INCREMENT
target_user_id / rated_by_user_id	`INT`	FK → users(id), CASCADE
score	`TINYINT`	CHECK (1–5)
review	`TEXT`	NULLABLE
created_at	`TIMESTAMP`	DEFAULT CURRENT_TIMESTAMP
—	`UNIQUE`	(target_user_id, rated_by_user_id)

reports

Column	Type	Constraints / Notes
id	`INT`	PK, AUTO_INCREMENT
request_id	`INT`	FK → help_requests(id), CASCADE

Column	Type	Constraints / Notes
reported_by_user_id	`INT`	FK → users(id), CASCADE
created_at	`TIMESTAMP`	DEFAULT CURRENT_TIMESTAMP
—	`UNIQUE`	(request_id, reported_by_user_id)

7. API Endpoints (CRUD Mapping)

All endpoints return JSON and enforce authentication/ownership checks server-side before touching the database. File uploads use `multipart/form-data` POST requests (with an `action` field distinguishing create/update/delete); all other writes use JSON request bodies.

Endpoint	Method	CRUD	Purpose
auth_api/register.php	POST	Create	Register a new user
auth_api/login.php	POST	Read	Authenticate & start session
auth_api/logout.php	POST	—	Destroy session
api/requests.php	GET	Read	List / filter / fetch one request
api/requests.php	POST	Create	action=create — new request
api/requests.php	POST	Update	action=update / action=resolve
api/requests.php	POST	Delete	action=delete
api/comments.php	GET/POST/PUT/DELETE	CRUD	Full CRUD on comments
api/messages.php	GET/POST/DELETE	CRUD	Thread read, send, delete own message
api/ratings.php	GET/POST	Create/Read	Read reviews; submit/upsert a rating
api/reports.php	POST	Create	Flag a request
api/notifications.php	GET/PUT/DELETE	Read/Update/Delete	Feed, mark read, clear
api/admin.php	GET/POST	Read/Update/Delete	Moderation actions (admin only)
api/profile.php	GET/POST	Read/Update	View / edit profile
api/donors.php	GET	Read	Blood donor directory

8. Security Measures

- **SQL Injection Prevention:** every query uses PDO prepared statements with bound parameters — no user input is ever concatenated into SQL.
- **Password Security:** passwords are hashed with `password_hash()` (bcrypt) and verified with `password_verify()`; plaintext passwords are never stored.
- **Session-based Authentication:** login state lives in server-side `$_SESSION`, with the session ID regenerated on login to prevent fixation.

- **Server-side Authorization:** ownership (and admin status) is re-checked on every update/delete request — the UI hiding a button is never the only protection.
- **File Upload Validation:** uploaded images are checked by real MIME type (`fileinfo`), size-limited, renamed to random filenames, and stored in a folder where PHP execution is disabled via `.htaccess`.
- **Output Escaping:** all user-supplied text is HTML-escaped on render to mitigate stored XSS.

9. Development Timeline

Week	Milestone
Week 1	Requirement analysis; ER diagram; schema design (schema.sql)
Week 2	Authentication module (register/login/logout/session) and base CRUD for help requests
Week 3	Comments, private messaging, notifications, image upload
Week 4	Ratings, reports/moderation, admin panel, donor directory, public profiles
Week 5	Testing, security review, UI polish, documentation, final report

10. Challenges & Learning Outcomes

- **Migrating off a NoSQL prototype:** the initial prototype used Firebase/Firestore; redesigning the data model into normalized MySQL tables required explicitly defining relationships (foreign keys) that Firestore's document structure had left implicit.
- **Authorization without a framework:** without a framework's built-in guards, every endpoint needed an explicit ownership/admin check, reinforcing the importance of defense-in-depth over relying on the UI.
- **Designing for cascading deletes:** using `ON DELETE CASCADE` correctly so that deleting a request or user cleans up related comments, messages, ratings, and reports without orphaned rows.
- **Replacing real-time listeners:** MySQL has no built-in push mechanism like Firestore's `onSnapshot`; the UI instead polls the API periodically to approximate real-time updates.

11. Conclusion

The Community Help Board demonstrates a complete, secure CRUD application built on PHP and MySQL: a normalized seven-table schema, a JSON REST API enforcing authentication and authorization server-side, and a responsive frontend that lets community members coordinate emergency assistance in real time. The project satisfies every stated requirement of the CSE 2208 DBMS Lab assignment — full CRUD operations against MySQL, a freely chosen web stack, and a feature set well beyond the minimum, including moderation, messaging, and a reviewer/rating system.

12. References & Repository

- PHP Manual — PDO: https://www.php.net/manual/en/book.pdo.php
- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- TailwindCSS Documentation: https://tailwindcss.com/docs

- Project GitHub Repository: github.com/rifat07yoo-hash/Community-Help-Board